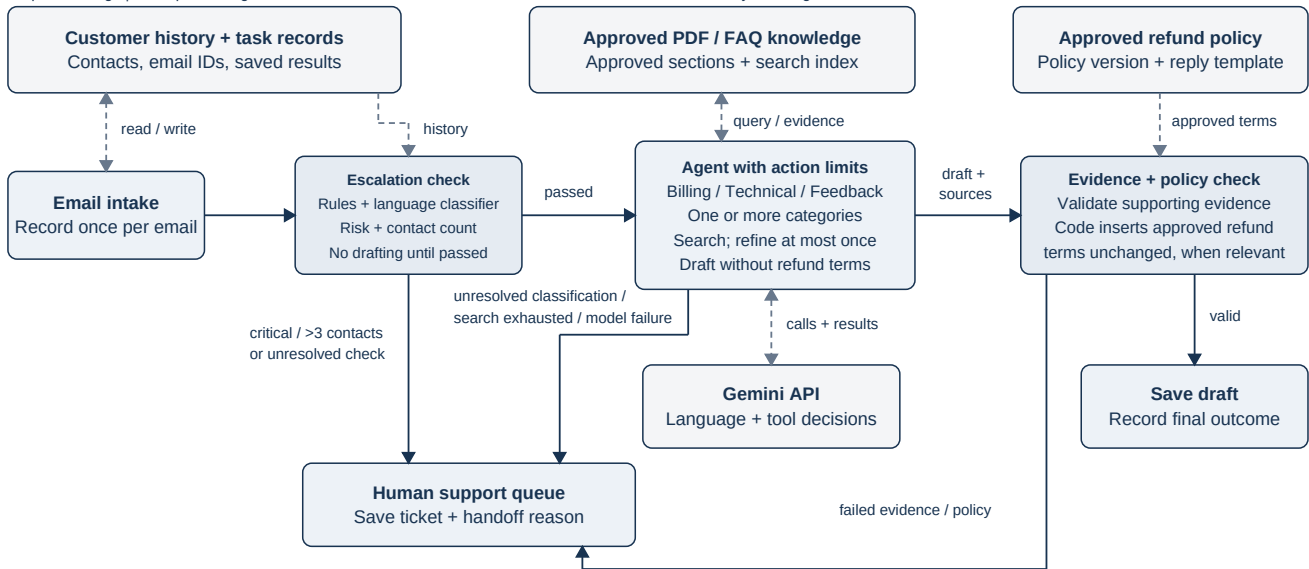


Agentic Architect Challenge

Jerence | Developer Intern assessment | Python + Gemini

Part 1 | Customer-support email architecture

Proposed design | Solid: processing flow. Dashed: data access / API calls. Two arrowheads indicate a two-way exchange.



Escalate BEFORE drafting: data loss, service outage, security breach, or >3 contacts in 7 days.

Assumptions: current email counts; unknown identity or missing history also requires a human. No automatic sending or refund approval.

Part 1: email system design

Before drafting, flag data loss, service outage, security breach, or more than three contacts in seven days. Missing history or uncertain risk sends the case to a human. Assumptions: the current email counts, both ends of the UTC time window are included, and retries keep the same message ID. Record contacts and final results safely to avoid duplicates. Escalated emails are categorised after handoff, by staff if needed, without drafting. The agent can improve a knowledge search once. When relevant, code inserts unchanged refund terms from approved, versioned policy fields and templates; missing or conflicting policy blocks the reply.

Part 2: website scraping and summaries

The assumed starting script misses JavaScript content and sends too much irrelevant text in one request. The replacement extracts main article content and tables, uses Playwright when little text is found, and limits HTML to 2 MiB. It processes up to 100 chunks and repeatedly summarises notes to keep each content input within 6,000 characters. One shortening request and a fixed word limit keep the final summary within 120 words. Long articles need more calls and time. Larger inputs are rejected. Login pages and infinite scrolling are not supported.

Part 3: document question-answering agent

The agent gives Gemini the full fictional OrbitDesk handbook and remembers twelve completed conversation turns. Gemini decides when to use the calculator; Python performs supported arithmetic and returns the result. Each question allows two tool calls and three model requests, excluding SDK retries. Document answers need a valid section and matching quote. Unsupported answers are withheld. Reset clears memory.

Trade-offs, failures and checks

Direct SDK calls keep the code simple; one short document does not need a vector database. Human handoffs reduce automation, while approved refund templates need maintenance. Each Gemini attempt has a 30-second timeout, with at most two attempts for selected transient failures; quota errors are not retried automatically. Failed chat turns are not saved. Part 3 offers verbose application logs, excluding prompts and keys. Offline checks cover processing, summary limits and JavaScript rendering; live checks evaluate Gemini's behaviour separately. A matching quote does not prove an answer is correct. Part 1 is a design, and no production load benchmark is claimed. Measure response times, quotas and queue waiting times before choosing how many workers run together.